Visão computacional para *pick-and-place* no FRET

Das imagens das câmeras à pose de preensão da bola:
modelo matemático, implementação e análise de erro

FRET — módulo `fret.vision`

Outubro de 2026

Resumo

Este documento descreve, com rigor matemático, o algoritmo de visão que o FRET usa para localizar uma bola sobre a mesa e transformar essa localização em alvos de preensão para os manipuladores OpenMANIPULATOR-X (OM-X) e OpenMANIPULATOR-Y (OMY). A cadeia completa é: renderização de duas câmeras calibradas no MuJoCo; segmentação por limiar no espaço HSV; filtragem morfológica; extração de contornos e seleção pela circularidade; estimação do centro pelo círculo envolvente mínimo; retroprojeção do pixel como um raio no mundo; interseção com o plano horizontal que contém o centro da bola; fusão das vistas por média ponderada; e cinemática inversa do ponto médio dos dedos da garra. Para cada etapa apresentamos a formulação, as hipóteses e proposições com demonstração (forma da silhueta da esfera projetada, viés de perspectiva, comportamento do círculo envolvente mínimo sob oclusão, limite digital da circularidade, jacobiano da elevação ao plano). Todos os números foram produzidos executando o código do repositório (`scripts/vision_algorithm_figures.py`), e as fórmulas fechadas foram verificadas numericamente contra a implementação.

Sumário

1 Visão geral **3**

2 Modelo de câmera **4**
2.1 Referenciais . . . 4
2.2 Projeção *pinhole* e intrínsecos . . . 4

3 Segmentação por cor no espaço HSV **5**
3.1 Conversão RGB → HSV (OpenCV, 8 bits) . . . 5
3.2 Limiar . . . 5

4 Filtragem morfológica **6**

5 Contornos, circularidade e centro **7**
5.1 Extração de contornos . . . 7
5.2 Área, perímetro e circularidade . . . 7
5.3 Filtros e seleção . . . 8
5.4 Centro pelo círculo envolvente mínimo . . . 8

6 Geometria da silhueta da esfera **8**

7 Do pixel ao mundo: elevação ao plano da mesa **10**
7.1 Retroprojeção . . . 10
7.2 O plano dos centros . . . 10
7.3 Fusão de múltiplas vistas . . . 11
7.4 Exemplo numérico completo . . . 11

8 Análise de erro **13**
8.1 Jacobiano da elevação . . . 13
8.2 Propagação de covariância e verificação de Monte Carlo . . . 13
8.3 Erro no modelo do plano . . . 13
8.4 Erro de calibração . . . 13
8.5 Oclusão parcial: estudo de caso . . . 14

9 Resultados na simulação **16**

10 Da posição da bola à pose de preensão **17**
10.1 Por que a posição basta . . . 17
10.2 Alvos cartesianos . . . 17
10.3 Cinemática inversa do ponto médio dos dedos . . . 17
10.4 Integração com a máquina de estados . . . 18

11 Limitações e melhorias possíveis **18**

A Parâmetros das configurações **19**

B Reprodutibilidade **19**

1 Visão geral

O problema resolvido pelo módulo `fret.vision` é o seguinte: dado um conjunto de imagens RGB de $N \geq 1$ câmeras com calibração conhecida, estimar a posição $\boldsymbol{p}^\star \in \mathbb{R}^3$, no referencial do mundo, do centro de uma bola de raio conhecido r apoiada sobre uma mesa horizontal de altura conhecida z_{mesa}. A posição estimada $\hat{\boldsymbol{p}}$ é então usada pelo controlador para calcular as configurações articulares de aproximação, preensão e elevação.

A Figura 1 resume o fluxo. O código está organizado em três estágios atrás de protocolos independentes do algoritmo (`fret/vision/protocols.py`): um *detector* (imagem → hipóteses em pixels), um *rastreador* opcional (não usado na configuração atual) e um *elevador de pose* (pixels → mundo). A orquestração está em `BallVisionPipeline.process` (`fret/vision/pipeline.py`), e a montagem concreta, em `fret/vision/factory.py`.

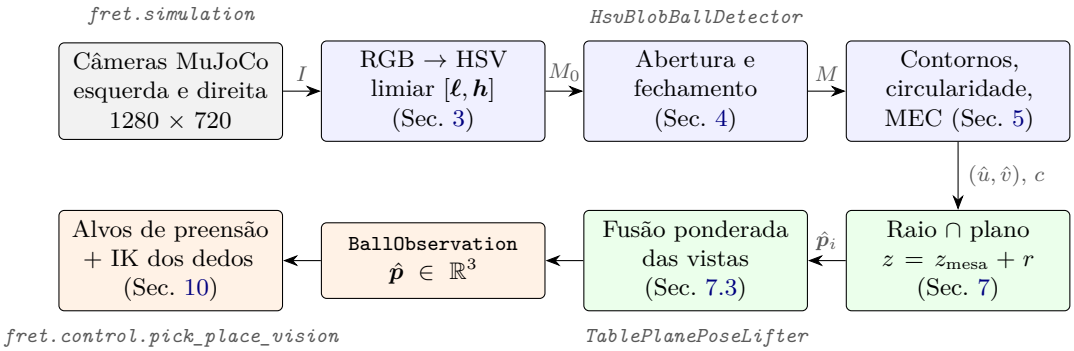


Figura 1: Fluxo completo, das imagens à preensão. Azul: detector em espaço de imagem; verde: geometria no mundo; laranja: consumidores no controle. Cada vista é processada de forma independente até a fusão.

Cenário. Nas células de *pick-and-place* (OM-X e OMY), duas câmeras ficam presas nos cantos superiores de um pórtico estrutural atrás do braço (`docs/vision/camera-layout.md`). A bola é amarelo-esverdeada (cor do material RGBA = (0,85; 0,92; 0,20; 1)) e tem raio 20 mm na célula OM-X e 43 mm na célula OMY. A Figura 2 mostra as duas vistas na pose de coleta padrão do OM-X, e a Figura 3, a estimativa final sobreposta à cena.

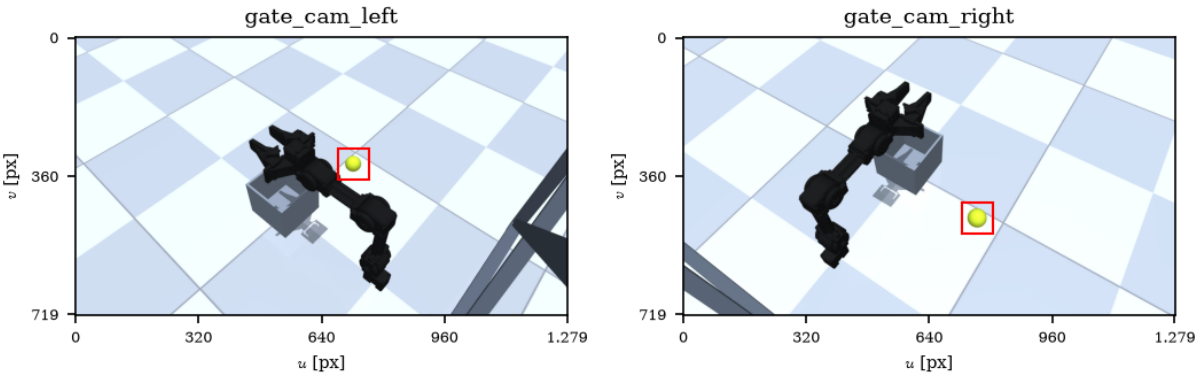
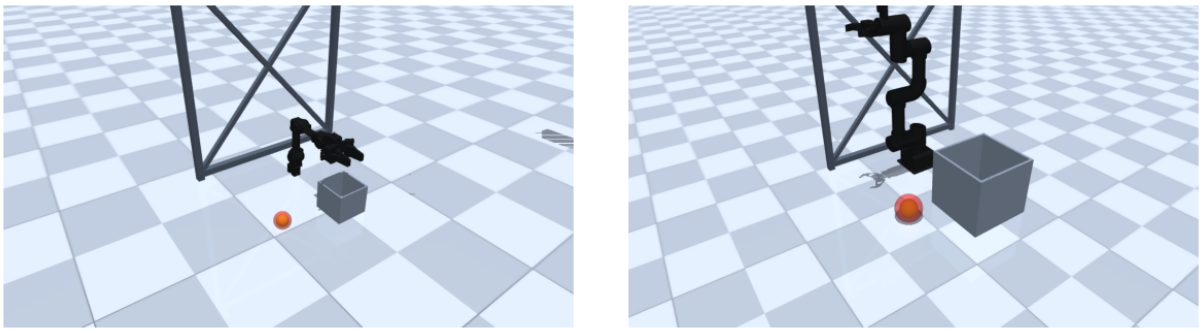


Figura 2: Imagens reais renderizadas pelas duas câmeras do pórtico (OM-X, bola em (0,22; −0,20) m). O quadrado vermelho marca a região da bola ampliada na Figura 5.

Notação. Vetores em negrito; $\boldsymbol{e}_z = (0, 0, 1)^\mathsf{T}$; $\tilde{\boldsymbol{u}} = (u, v, 1)^\mathsf{T}$ são coordenadas homogêneas de pixel; $[\,\cdot\,]_{xy}$ é a projeção nas duas primeiras componentes. Ângulos em radianos, salvo indicação. Os índices E e D referem-se às câmeras esquerda (`gate_cam_left`) e direita (`gate_cam_right`).



(a) OM-X **(b)** OMY

Figura 3: Vista geral das células. A esfera vermelha translúcida (`cv_ball_ghost`, apenas visual) é posicionada na estimativa de visão $\hat{\boldsymbol{p}}$; a bola amarela real fica dentro dela.

2 Modelo de câmera

2.1 Referenciais

Usamos três referenciais: o mundo $\{W\}$ (origem na base do braço, x para frente, z para cima); a câmera do MuJoCo $\{M\}$, que olha ao longo de $-z_M$ com y_M para cima na imagem; e a câmera óptica no padrão do OpenCV $\{C\}$, com x_C para a direita, y_C para baixo e z_C para dentro da cena. Todo o pipeline trabalha em $\{C\}$.

O MuJoCo fornece a posição $\boldsymbol{O} \in \mathbb{R}^3$ do centro óptico (`cam_xpos`) e a matriz $R_M \in SO(3)$ (`cam_xmat`), cujas colunas são os eixos de $\{M\}$ expressos em $\{W\}$. Como $x_C = x_M$, $y_C = -y_M$ e $z_C = -z_M$, temos

$$R \;=\; R_M \,\mathrm{diag}(1,-1,-1), \qquad T_{WC} = \begin{bmatrix} R & \boldsymbol{O} \\ \mathbf{0}^{\mathsf{T}} & 1 \end{bmatrix}, \tag{1}$$

implementado em `mujoco_xmat_to_opencv_rotation` (`fret/simulation/mujoco_camera.py`). R continua sendo uma rotação: é produto de matrizes ortogonais e $\det R = \det R_M \cdot (1)(-1)(-1) = 1$. A transformação leva coordenadas da câmera para o mundo: $\boldsymbol{x}_W = R\,\boldsymbol{x}_C + \boldsymbol{O}$.

2.2 Projeção *pinhole* e intrínsecos

Com a matriz de intrínsecos

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \tag{2}$$

um ponto $\boldsymbol{x}_W$ à frente da câmera é projetado em

$$\boldsymbol{x}_C = R^{\mathsf{T}}(\boldsymbol{x}_W - \boldsymbol{O}) = (X,Y,Z)^{\mathsf{T}}, \qquad u = f_x\frac{X}{Z} + c_x, \quad v = f_y\frac{Y}{Z} + c_y, \quad Z > 0. \tag{3}$$

O simulador não tem distorção de lente, e o tipo `CameraIntrinsics` deliberadamente omite coeficientes de distorção.

O MuJoCo define a câmera pelo campo de visão vertical ϕ_y (`fovy`). No plano $Z = 1$, a meia-altura visível é $\tan(\phi_y/2)$ e corresponde a $H/2$ pixels; com pixels quadrados,

$$f_x = f_y = f = \frac{H/2}{\tan(\phi_y/2)}, \qquad c_x = \frac{W-1}{2}, \quad c_y = \frac{H-1}{2}, \tag{4}$$

(`intrinsics_from_fovy`). O ponto principal usa a convenção de que o pixel de índice (i,j) tem centro nas coordenadas inteiras (i,j), a mesma do OpenCV para contornos. Para $H = 720$: com $\phi_y = 42°$ (OM-X), $f = 360/\tan(21°) = 937{,}8321$ px; com $\phi_y = 45°$ (OMY), $f = 360/\tan(22{,}5°) = 869{,}1169$ px. Em ambos os casos, $(c_x, c_y) = (639{,}5; 359{,}5)$.

Consistência da calibração. As matrizes K e T_{WC} de cada câmera ficam guardadas nos arquivos de configuração `omx_portal_overhead.yml` e `omy_portal_overhead.yml`. Verificamos contra o modelo MuJoCo carregado: a maior diferença entre os T_{WC} do YAML e os calculados a partir do modelo foi $4{,}4 \times 10^{-11}$ (OM-X) e $4{,}9 \times 10^{-11}$ (OMY); a diferença em f e c foi $\leq 4{,}6 \times 10^{-8}$ px; e $\max |R^{\mathsf{T}} R - I| \leq 1{,}3 \times 10^{-10}$, com $|\det R - 1| \leq 7{,}6 \times 10^{-11}$. Portanto, a calibração usada pelo pipeline coincide com a geometria renderizada até o arredondamento do YAML.

3 Segmentação por cor no espaço HSV

3.1 Conversão RGB → HSV (OpenCV, 8 bits)

Seja $I : \Omega \to \{0, \ldots, 255\}^3$ a imagem RGB, com $\Omega = \{0, \ldots, W-1\} \times \{0, \ldots, H-1\}$. Para um pixel (R, G, B), sejam $M_{\max} = \max(R, G, B)$, $m = \min(R, G, B)$ e $\Delta = M_{\max} - m$. O OpenCV calcula

$$V = M_{\max}, \qquad S = \begin{cases} 255\,\Delta / M_{\max}, & M_{\max} > 0 \\ 0, & M_{\max} = 0 \end{cases} \tag{5}$$

$$H' = \begin{cases} 60\,(G-B)/\Delta, & M_{\max} = R \\ 120 + 60\,(B-R)/\Delta, & M_{\max} = G \\ 240 + 60\,(R-G)/\Delta, & M_{\max} = B \end{cases} \quad (H' \leftarrow H' + 360 \text{ se } H' < 0), \qquad H = H'/2, \tag{6}$$

com arredondamento para inteiro. Assim $H \in \{0, \ldots, 179\}$ representa o matiz em graus divididos por dois, e $S, V \in \{0, \ldots, 255\}$. O código converte RGB→BGR→HSV (`_to_bgr` e `cv2.cvtColor`).

Exemplo. A cor difusa do material da bola, (217; 235; 51), tem $M_{\max} = G = 235$, $m = 51$ e $\Delta = 184$. Logo $V = 235$, $S = 255 \cdot 184/235 \approx 199{,}7 \to 200$ e $H' = 120 + 60(51 - 217)/184 \approx 65{,}9$, ou seja, $H = 33$. A chamada `cv2.cvtColor` retorna exatamente (33; 200; 235).

3.2 Limiar

Definição 1 (Máscara de cor)**.** Dados limites $\boldsymbol{\ell}, \boldsymbol{h} \in \{0, \ldots, 255\}^3$, a máscara inicial é

$$M_0(\boldsymbol{x}) = \begin{cases} 255, & \boldsymbol{\ell} \preceq \mathrm{hsv}(I(\boldsymbol{x})) \preceq \boldsymbol{h} \\ 0, & \text{caso contrário,} \end{cases} \tag{7}$$

em que $\preceq$ é a ordem componente a componente, com limites inclusivos (`cv2.inRange`). Se a configuração define um segundo intervalo $[\boldsymbol{\ell}', \boldsymbol{h}']$, usa-se $M_0 \leftarrow M_0 \vee M_0'$. Isso existe porque o matiz é circular: um objeto vermelho ocupa $H \approx 0$ e $H \approx 179$ e precisa de dois intervalos.

Nas células com pórtico, $\boldsymbol{\ell} = (20; 40; 40)$ e $\boldsymbol{h} = (50; 255; 255)$, sem segundo intervalo. A Figura 4 justifica a escolha com dados: os pixels da bola, identificados pela renderização de segmentação do MuJoCo (a verdade de campo), concentram-se em $H \in [30; 36]$ (percentis 1 e 99), com mediana $(H, S, V) = (34; 196; 231)$. O piso e o pórtico ficam em $H \approx 90$–120 ou têm S baixo. Na vista da pose padrão, 100,00 % dos pixels da bola caem dentro da caixa de limiar e 0,00 % de uma amostra de 80.000 pixels de fundo caem dentro dela (OMY: 99,98 % e 0,01 %). O valor V mínimo da bola (130, no lado sombreado) está bem acima de $\ell_V = 40$, o que dá margem para variação de iluminação.

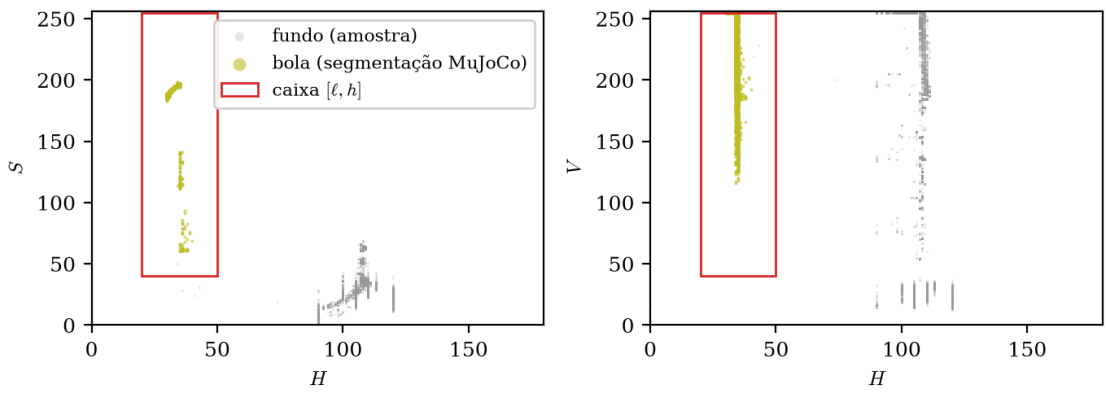


Figura 4: Distribuição dos pixels da bola (verde-oliva, rótulos da segmentação do MuJoCo) e de uma amostra do fundo (cinza) nos planos H–S e H–V, com a caixa de limiar $[\boldsymbol{\ell}, \boldsymbol{h}]$ da configuração OM-X. As duas câmeras estão incluídas.

4 Filtragem morfológica

Identificamos a máscara binária com o conjunto $X = \{\boldsymbol{x} \in \Omega : M_0(\boldsymbol{x}) \neq 0\} \subset \mathbb{Z}^2$. Seja $B \subset \mathbb{Z}^2$ um elemento estruturante simétrico ($B = -B$) centrado na origem e $B_{\boldsymbol{x}} = \boldsymbol{x} + B$.

Definição 2. Erosão $X \ominus B = \{\boldsymbol{x} : B_{\boldsymbol{x}} \subseteq X\}$; dilatação $X \oplus B = \{\boldsymbol{x} : B_{\boldsymbol{x}} \cap X \neq \emptyset\}$; abertura $X \circ B = (X \ominus B) \oplus B$; fechamento $X \bullet B = (X \oplus B) \ominus B$.

O detector aplica $M = (M_0 \circ B) \bullet B$ (`mask_for_frame`), com o elemento estruturante devolvido por

`cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))` :

$$B = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

Proposição 1 (Propriedades usadas)**.** *(i)* $X \circ B = \bigcup\{B_{\boldsymbol{y}} : B_{\boldsymbol{y}} \subseteq X\} \subseteq X$*, ou seja, a abertura mantém exatamente os pontos cobertos por alguma translação de* B *contida em* X*. (ii)* $X \subseteq X \bullet B$ *e* $(X \bullet B)^c = X^c \circ B$*. (iii) As duas operações são crescentes e idempotentes.*

Demonstração. (i) $\boldsymbol{z} \in (X \ominus B) \oplus B$ se e só se existe $\boldsymbol{y}$ com $B_{\boldsymbol{y}} \subseteq X$ e $\boldsymbol{z} \in B_{\boldsymbol{y}}$ (simetria de B). (ii) Pela dualidade $(X \oplus B)^c = X^c \ominus B$, temos $(X \bullet B)^c = (X^c \ominus B) \oplus B = X^c \circ B$; a inclusão segue de (i) aplicado a X^c. (iii) É o resultado clássico de Matheron e Serra [2]. □

Consequência prática: pela Proposição 1(i), a abertura apaga qualquer componente de falso positivo que não contenha uma cópia de B (pontos isolados, linhas de 1 a 2 px de largura em bordas de objetos). Pelo item (ii), o fechamento preenche furos e reentrâncias do complemento que não contêm B, como o reflexo especular ou o recorte deixado por uma aresta fina à frente da bola. Para a bola sem oclusão, de raio $\approx$ 20–36 px, as duas operações quase não alteram a silhueta (Figura 5).

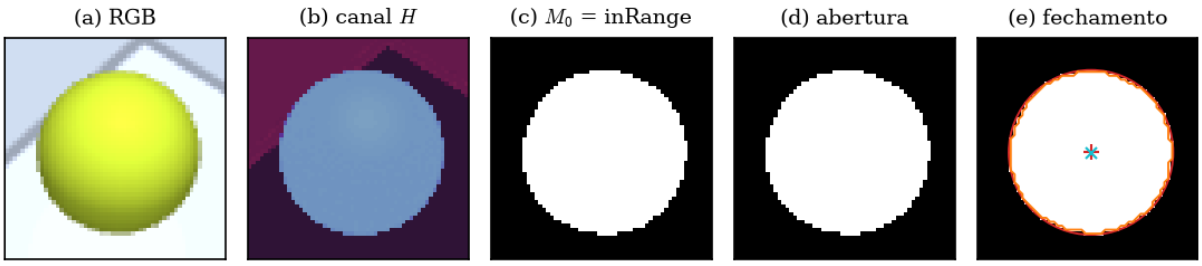


Figura 5: Etapas do detector na câmera esquerda (OM-X, pose padrão), recorte de 56×56 px. (e) Contorno externo (laranja), círculo envolvente mínimo (vermelho) e centro estimado (+). O × ciano é a projeção exata do centro da bola pela Eq. (3).

5 Contornos, circularidade e centro

5.1 Extração de contornos

O detector extrai as bordas externas das componentes 8-conexas de M com o algoritmo de seguimento de bordas de Suzuki e Abe [3] (`cv2.findContours`, modo `RETR_EXTERNAL`). Cada contorno é um polígono fechado $\mathcal{P} = (\boldsymbol{q}_0, \ldots, \boldsymbol{q}_{n-1})$ cujos vértices são centros de pixels de borda. O modo `CHAIN_APPROX_SIMPLE` só remove vértices intermediários de trechos horizontais, verticais ou diagonais, de modo que o polígono (como subconjunto do plano) não muda.

5.2 Área, perímetro e circularidade

$$A(\mathcal{P}) = \frac{1}{2}\left|\sum_{k=0}^{n-1}(x_k y_{k+1} - x_{k+1} y_k)\right|, \qquad P(\mathcal{P}) = \sum_{k=0}^{n-1}\|\boldsymbol{q}_{k+1} - \boldsymbol{q}_k\|, \qquad c = \frac{4\pi A}{P^2}, \tag{8}$$

com índices módulo n (`contourArea`, `arcLength`, `_circularity`). Como os vértices estão sobre a rede $\mathbb{Z}^2$, o teorema de Pick dá, para um polígono simples, $A = I + B_\partial/2 - 1$, com I pontos interiores e B_∂ pontos de rede na borda. Logo, A fica cerca de meia camada de pixels abaixo da contagem de pixels da componente. No exemplo da Seção 7.4, $A = 1305{,}0$ contra 1342 pixels visíveis da bola na segmentação de referência e $1353{,}5\,\text{px}^2$ de área teórica da silhueta.

Pela desigualdade isoperimétrica, $P^2 \geq 4\pi A$ para qualquer curva plana fechada, com igualdade somente para o círculo. Portanto $c \in (0, 1]$ e $c = 1$ caracteriza o disco no contínuo. No caso digital, o máximo atingível é menor:

Proposição 2 (Limite digital da circularidade)**.** *Seja $\mathcal{P}_R$ o contorno externo de um disco digitalizado de raio R. Quando $R \to \infty$,*

$$\frac{P(\mathcal{P}_R)}{2\pi R} \;\to\; \frac{8}{\pi}(\sqrt{2} - 1) \approx 1{,}05479, \qquad c(\mathcal{P}_R) \;\to\; c_\infty = \left(\frac{\pi}{8(\sqrt{2} - 1)}\right)^2 \approx 0{,}8988. \tag{9}$$

Demonstração. Por simetria, basta um octante. Nele, a tangente ao círculo faz ângulo $\theta \in [0, \pi/4]$ com o eixo dominante, e o contorno é um caminho 8-conexo de passos axiais (comprimento 1) e diagonais ($\sqrt{2}$). Por unidade de avanço no eixo dominante, o caminho dá $\tan\theta$ passos diagonais e $1 - \tan\theta$ passos axiais. Como um elemento de arco ds avança $\cos\theta\, ds$, o comprimento do caminho por unidade de arco é $\cos\theta\,[(1 - \tan\theta) + \sqrt{2}\tan\theta] = \cos\theta + (\sqrt{2} - 1)\sin\theta$. No círculo, θ é proporcional ao comprimento de arco; a média no octante é

$$\frac{4}{\pi}\int_0^{\pi/4}(\cos\theta + (\sqrt{2} - 1)\sin\theta)\, d\theta = \frac{4}{\pi}(2\sqrt{2} - 2) = \frac{8}{\pi}(\sqrt{2} - 1).$$

O erro relativo de área é $O(P/R^2) = O(1/R)$, de modo que $A/(\pi R^2) \to 1$, e o limite de c segue de (8). □

A Figura 7(a) confirma a proposição numericamente com o mesmo código OpenCV: a circularidade média de discos digitais é 0,884 em $R = 5\,\text{px}$, 0,898 em $R = 30\,\text{px}$ e 0,898 em $R = 60\,\text{px}$, ou seja, c_∞ com três casas. Isso explica por que o limiar configurado, $c_{\min} = 0{,}45$ (pórtico) ou 0,55 (fixture), está bem abaixo de 1. A silhueta de uma esfera é uma elipse levemente alongada (Seção 6). Mesmo no canto da imagem, com razão de eixos $b/a \approx \cos\alpha \approx 0{,}76$, a circularidade contínua dessa elipse é $\approx 0{,}97$, o que ainda deixa grande margem até $c_{\min}$.

5.3 Filtros e seleção

Um contorno é aceito se $A_{\min} \le A \le A_{\max}$ e $c \ge c_{\min}$. Entre os aceitos, o detector escolhe

$$k^\star = \arg\max_k \; s_k, \qquad s_k = c_k + 0{,}05\,\min\!\big(1, A_k/A_{\max}\big), \tag{10}$$

e retorna *no máximo uma* detecção por câmera, com confiança $\kappa = \mathrm{clip}(c_{k^\star}, 0, 1)$. Com $A_{\max} = 500.000\,\text{px}^2$ e bolas de $\lesssim 4000\,\text{px}^2$, o termo de área vale menos que 4×10^{-4}, ou seja, a seleção é, na prática, pela maior circularidade, e a área só desempata.

5.4 Centro pelo círculo envolvente mínimo

Definição 3. O círculo envolvente mínimo (MEC) de um conjunto compacto $S \subset \mathbb{R}^2$ é o disco fechado de menor raio que contém S. Ele existe e é único [4].

O detector define o centro $(\hat{u}, \hat{v})$ e o raio $\hat{\rho}$ da bola na imagem como o centro e o raio do MEC dos vértices do contorno escolhido (`cv2.minEnclosingCircle`). A implementação do OpenCV é exata nas detecções reais da Seção 9: a maior distância dos vértices ao centro retornado difere do raio retornado em no máximo $1{,}0 \times 10^{-4}\,\text{px}$ (OM-X) e $1{,}0 \times 10^{-4}\,\text{px}$ (OMY). As duas proposições a seguir explicam o que esse centro mede.

6 Geometria da silhueta da esfera

Proposição 3 (Silhueta da esfera projetada)**.** *Seja uma esfera de raio r cujo centro, em $\{C\}$, está à distância D do centro óptico e faz ângulo α com o eixo óptico. Sejam $\beta = \arcsin(r/D)$ e $\alpha + \beta < \pi/2$. Em coordenadas normalizadas ($Z = 1$), rotacionadas em torno do eixo óptico para que o centro projetado fique no semieixo $x > 0$, a silhueta é a elipse*

$$\frac{(x - x_0)^2}{a^2} + \frac{y^2}{b^2} = 1, \quad x_0 = \frac{\sin\alpha\cos\alpha}{\mathcal{A}},\; a = \frac{\sin\beta\cos\beta}{\mathcal{A}},\; b = \frac{\sin\beta}{\sqrt{\mathcal{A}}},\; \mathcal{A} = \cos^2\beta - \sin^2\alpha. \tag{11}$$

Em particular, o centro da elipse é deslocado radialmente para fora em relação à projeção do centro da esfera, $\tan\alpha$, por

$$\delta = x_0 - \tan\alpha = \frac{\tan\alpha\,\sin^2\beta}{\cos^2\beta - \sin^2\alpha} \;\ge 0. \tag{12}$$

Demonstração. Os raios tangentes à esfera formam um cone circular de eixo $\hat{\boldsymbol{c}} = (\sin\alpha, 0, \cos\alpha)$ e semiabertura β: um vetor $\boldsymbol{d} = (x, y, 1)$ está no cone se e só se $(\boldsymbol{d}\cdot\hat{\boldsymbol{c}})^2 = \cos^2\beta\,\|\boldsymbol{d}\|^2$, isto é,

$$(x\sin\alpha + \cos\alpha)^2 = \cos^2\beta\,(x^2 + y^2 + 1).$$

Reorganizando, $\mathcal{A}x^2 - 2x\sin\alpha\cos\alpha + \cos^2\beta\,y^2 = \cos^2\alpha - \cos^2\beta$. Completando o quadrado em x,

$$\mathcal{A}(x - x_0)^2 + \cos^2\beta\,y^2 = \cos^2\alpha - \cos^2\beta + \frac{\sin^2\alpha\cos^2\alpha}{\mathcal{A}} = \frac{\cos^2\beta\sin^2\beta}{\mathcal{A}},$$

em que a última igualdade vem da expansão do numerador, $(\cos^2\alpha - \cos^2\beta)(\cos^2\beta - \sin^2\alpha) + \sin^2\alpha\cos^2\alpha = \cos^2\beta(1 - \cos^2\beta)$. Dividindo pelo lado direito, obtêm-se a e b, e $\mathcal{A} > 0$ porque $\alpha + \beta < \pi/2$. Para o viés, $x_0 - \tan\alpha = \sin\alpha(\cos^2\alpha - \mathcal{A})/(\mathcal{A}\cos\alpha)$ e $\cos^2\alpha - \mathcal{A} = 1 - \cos^2\beta = \sin^2\beta$. □

Corolário 4. *Sem oclusão, e no limite contínuo, o MEC da silhueta é o círculo de centro* $(x_0, 0)$ *e raio* a*. Portanto o detector estima o centro da* elipse*, com viés sistemático de* $f\delta$ *pixels na direção radial, e* $\hat{\rho} \approx fa$*.*

Demonstração. A elipse está contida no disco de centro $(x_0, 0)$ e raio a (pois $b \le a$, o que equivale a $\mathcal{A} \le \cos^2 \beta$). Os vértices $(x_0 \pm a, 0)$ são antípodas nesse disco, e qualquer disco que os contenha tem raio $\ge a$. Pela unicidade do MEC, ele é esse disco. □

A Figura 6 mostra $f\delta$ em função de α. As curvas analíticas coincidem com a projeção numérica de 20.001 pontos do círculo de tangência (diferença máxima $< 10^{-3}$ px). Nas geometrias reais, o viés máximo, no canto da imagem ($\alpha \approx 38{,}1°$ e $40{,}2°$), é 0,573 px (OM-X) e 1,390 px (OMY). No exemplo da Seção 7.4, a Eq. (12) prevê 0,043 e 0,116 px, e os erros de centro medidos são 0,05 e 0,22 px. Os raios previstos, $fa = 20{,}80$ e 25,23 px, conferem com os medidos, $\hat{\rho} = 20{,}88$ e 25,21 px. Como cada pixel equivale a ≈ 1 mm no plano da mesa (Seção 8), esse viés contribui com décimos de milímetro. Ele poderia ser removido invertendo a Eq. (12), mas hoje não é corrigido.

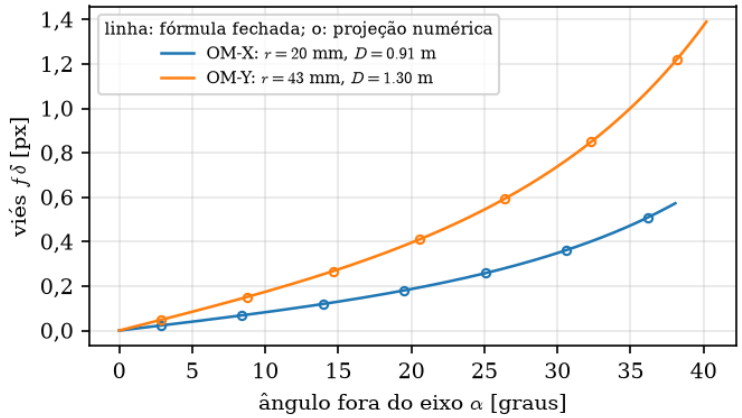


Figura 6: Viés de perspectiva $f\delta$ (Eq. 12) entre o centro da silhueta e a projeção do centro da esfera, na distância da pose padrão. Os círculos são a verificação numérica independente.

Proposição 5 (MEC sob oclusão)**.** *Seja* $\mathcal{D}$ *o disco da silhueta (raio* R*, centro* $\boldsymbol{c}$*) e* $S \subseteq \mathcal{D}$ *a parte visível, compacta.*

(i) Se S *contém dois pontos antípodas de* $\partial\mathcal{D}$ *(por exemplo, um arco de borda com ângulo* $\ge \pi$*), então* $\mathrm{MEC}(S) = \mathcal{D}$ *e o centro estimado é exato.*

(ii) Se o oclusor é um semiplano e a parte visível é menor que meio disco, $S = \mathcal{D} \cap \{\boldsymbol{x} : \langle \boldsymbol{x} - \boldsymbol{c}, \boldsymbol{n} \rangle \le -h\}$ *com* $0 < h < R$ *e* $\|\boldsymbol{n}\| = 1$*, então* $\mathrm{MEC}(S)$ *é o disco que tem a corda como diâmetro: centro* $\boldsymbol{c} - h\boldsymbol{n}$*, raio* $\sqrt{R^2 - h^2}$*. O erro de centro é* h*, com fração visível* $f(h) = \left[\arccos(h/R) - (h/R)\sqrt{1 - h^2/R^2}\right]/\pi$*.*

Demonstração. (i) Todo disco que contém dois pontos à distância $2R$ tem raio $\ge R$, e $\mathcal{D} \supseteq S$ tem raio R. Pela unicidade, $\mathrm{MEC}(S) = \mathcal{D}$. (ii) Escreva $\boldsymbol{x} \in S$ como $\boldsymbol{x} = \boldsymbol{c} - s\boldsymbol{n} + w\boldsymbol{\tau}$, com $\boldsymbol{\tau} \perp \boldsymbol{n}$, $s \ge h$ e $s^2 + w^2 \le R^2$. Então $\|\boldsymbol{x} - (\boldsymbol{c} - h\boldsymbol{n})\|^2 = (s-h)^2 + w^2 \le (s-h)^2 + R^2 - s^2 = R^2 - h^2 - 2h(s-h) \le R^2 - h^2$. Logo, o disco proposto contém S. As extremidades da corda, $s = h$ e $w = \pm\sqrt{R^2 - h^2}$, são antípodas nele, e o argumento de (i) conclui. □

A Proposição 5 mostra a vantagem do MEC sobre o centróide (momentos de ordem 1). Com até metade do disco oculta por uma borda reta, o MEC continua exato, enquanto o centróide de meio disco já erra $4R/(3\pi) \approx 0{,}42R$. A Figura 7(b) compara os dois para um disco digital de $R = 30$ px. No regime (i), o erro medido do MEC fica $\le 0{,}094R$. O desvio residual em relação à teoria vem do condicionamento: perto da corda mais larga, a distância aos pontos extremos varia

só em segunda ordem com a posição do centro, e uma perturbação de digitalização ε desloca o centro em cerca de $\sqrt{2R\varepsilon}$ (estimativa heurística; para $\varepsilon \approx 0{,}07$ px, ≈ 2 px). A circularidade cai monotonamente com a oclusão (curva traço-ponto). Isso justifica, em parte, usá-la como confiança.

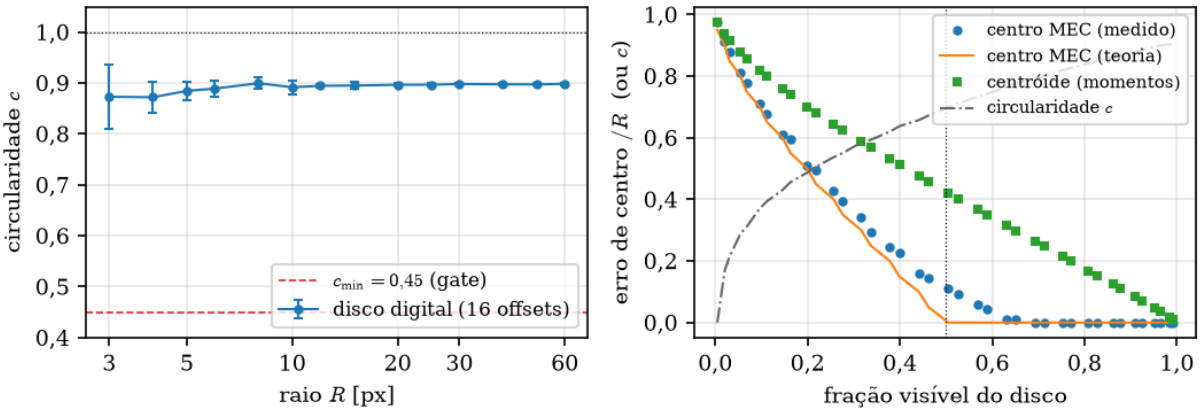


Figura 7: (a) Circularidade de discos digitais em função do raio (média e desvio padrão sobre 16 deslocamentos subpixel); a assíntota é $c_\infty \approx 0{,}899$ (Proposição 2). (b) Oclusão por semiplano de um disco com $R = 30$ px: erro do centro do MEC (medido e teórico, Proposição 5), erro do centróide e circularidade.

7 Do pixel ao mundo: elevação ao plano da mesa

7.1 Retroprojeção

O pixel $(\hat{u}, \hat{v})$ define o raio

$$\boldsymbol{x}(\lambda) = \boldsymbol{O} + \lambda\, \boldsymbol{d}, \qquad \boldsymbol{d} = R\, K^{-1}\tilde{\boldsymbol{u}} = R \begin{bmatrix} (\hat{u} - c_x)/f_x \\ (\hat{v} - c_y)/f_y \\ 1 \end{bmatrix}, \qquad \lambda > 0. \tag{13}$$

O código normaliza $\boldsymbol{d}$ (`_pixel_ray_cam`). Isso muda o parâmetro λ, mas não o ponto de interseção. Com a forma não normalizada de (13), como a terceira componente de $K^{-1}\tilde{\boldsymbol{u}}$ é 1, o valor de λ na interseção é igual à profundidade Z do ponto em $\{C\}$.

7.2 O plano dos centros

Uma esfera de raio r apoiada num plano horizontal de altura z_{mesa} tem o centro em $z = z_{\text{mesa}} + r$. O lifter usa o plano

$$\Pi = \{\boldsymbol{x} \in \mathbb{R}^3 :\ \boldsymbol{e}_z^{\mathsf{T}}\boldsymbol{x} = z_\Pi\}, \qquad z_\Pi = z_{\text{mesa}} + r, \tag{14}$$

com $z_{\text{mesa}} = 0$ e $r = 20$ mm (OM-X) ou 43 mm (OMY), lidos de `lifter` no YAML. A interseção é

$$\lambda^\star = \frac{z_\Pi - O_z}{d_z}, \qquad \hat{\boldsymbol{p}} = \boldsymbol{O} + \lambda^\star \boldsymbol{d}, \tag{15}$$

definida quando $|d_z| \geq 10^{-12}$ (raio não paralelo ao plano) e $\lambda^\star > 0$ (plano à frente da câmera). Caso contrário, a vista é descartada (`intersect_ray_horizontal_plane`).

Proposição 6 (Exatidão do modelo)**.** *Se o centro verdadeiro $\boldsymbol{p}^\star$ está em Π, está à frente da câmera e $(\hat{u}, \hat{v}) = \pi(\boldsymbol{p}^\star)$ é sua projeção exata pela Eq. (3), então a Eq. (15) devolve $\hat{\boldsymbol{p}} = \boldsymbol{p}^\star$.*

Demonstração. Pela definição de projeção central, $\boldsymbol{p}^\star = \boldsymbol{O} + Z^\star \boldsymbol{d}$ com $\boldsymbol{d}$ dado por (13) e $Z^\star > 0$. Como $d_z \neq 0$, a reta encontra Π em um único ponto, que portanto é $\boldsymbol{p}^\star$. □

Assim, com uma única câmera, o erro de posição vem apenas de três fontes: erro no pixel do centro (viés da Proposição 3, oclusão, digitalização), erro no modelo do plano (z_Π) e erro de calibração. Essas fontes são quantificadas na Seção 8. Uma consequência importante é que a altura estimada é sempre z_Π por construção, de modo que o erro em z é nulo enquanto a bola de fato está sobre a mesa.

7.3 Fusão de múltiplas vistas

Para cada câmera i configurada que produziu uma detecção, o lifter fica com a de maior confiança (o detector já retorna no máximo uma), calcula $\hat{\boldsymbol{p}}_i$ por (15) e funde

$$\hat{\boldsymbol{p}} = \sum_i w_i \, \hat{\boldsymbol{p}}_i, \qquad w_i = \frac{\max(\kappa_i, 10^{-3})}{\sum_j \max(\kappa_j, 10^{-3})}, \tag{16}$$

e reimpõe $\hat{p}_z = z_\Pi$ (`TablePlanePoseLifter.lift`). Como Π é afim e (16) é uma combinação convexa de pontos de Π, a reimposição só remove arredondamento. Basta uma vista: se uma câmera estiver obstruída, a outra sozinha define $\hat{\boldsymbol{p}}$.

Observação 1 (Pesos ótimos). Se os $\hat{\boldsymbol{p}}_i$ fossem não viesados e independentes, com covariâncias Σ_i no plano, o estimador linear de variância mínima seria $\hat{\boldsymbol{p}} = (\sum_i \Sigma_i^{-1})^{-1} \sum_i \Sigma_i^{-1} \hat{\boldsymbol{p}}_i$. A Eq. (16) usa pesos escalares baseados na circularidade, e não em Σ_i. Os dois coincidem quando as vistas têm covariâncias iguais. Sem oclusão, $\kappa_E \approx \kappa_D \approx 0{,}89$ e os pesos ficam $\approx \frac{1}{2}$.

A saída é um `BallObservation` com `position_world`$= \hat{\boldsymbol{p}}$, `radius_m`$= r$, `timestamp` igual ao maior carimbo de tempo entre os quadros usados e `source` igual ao identificador do pipeline. O campo `covariance` existe no tipo, mas o lifter atual não o preenche (Seção 8 mostra como calculá-lo), e `pickable` permanece `None`.

7.4 Exemplo numérico completo

A Tabela 1 acompanha o cálculo na célula OM-X com a bola na pose de coleta padrão, $\boldsymbol{p}^\star = (220{,}00; -200{,}00; 20{,}00)$ mm. Todos os valores foram obtidos do pipeline real; a reimplementação passo a passo usada para preencher a tabela reproduz a máscara do detector bit a bit e a posição fundida com tolerância de 10^{-12} m.

Note que $\lambda^\star$ coincide com a profundidade Z, como previsto. Na célula OMY, na pose padrão $(350{,}00; -300{,}00)$ mm, o mesmo cálculo dá $\hat{\boldsymbol{p}} = (349{,}79; -300{,}36; 43{,}00)$ mm, com erro horizontal de 0,418 mm. A Figura 8 mostra os raios e os erros no plano.

Tabela 1: Exemplo completo, OM-X, pose `pick_default`. Comprimentos em metros, salvo indicação.

	câmera esquerda (E)	câmera direita (D)
contornos encontrados	1	1
área A / perímetro P [px]	1305,0 / 135,88	1898,5 / 162,61
circularidade $c = \kappa$	0,8882	0,9022
IoU máscara × segmentação	0,984	0,988
centro MEC $(\hat{u}, \hat{v})$ [px]	(722,37; 327,41)	(764,56; 468,25)
projeção exata $\pi(\boldsymbol{p}^\star)$ [px]	(722,34; 327,45)	(764,63; 468,04)
erro de centro [px]	0,05	0,22
viés previsto $f\delta$ (Eq. 12) [px]	0,043	0,116
raio MEC $\hat{\rho}$ / previsto fa [px]	20,88 / 20,80	25,21 / 25,23
$K^{-1}\tilde{\boldsymbol{u}}$	(0,08836; −0,03421; 1,00000)	(0,13335; 0,11596; 1,00000)
$\boldsymbol{d} = RK^{-1}\tilde{\boldsymbol{u}}$	(0,37532; −0,55199; −0,75062)	(0,45006; 0,13238; −0,90064)
centro óptico $\boldsymbol{O}$	(−0,120; 0,300; 0,700)	(−0,120; −0,300; 0,700)
$\lambda^\star = (z_\Pi - O_z)/d_z$	0,9059	0,7550
profundidade Z de $\boldsymbol{p}^\star$	0,9059	0,7551
$\hat{\boldsymbol{p}}_i$ [mm]	(220,01; −200,06; 20,00)	(219,80; −200,05; 20,00)
peso w_i	0,4961	0,5039
$\hat{\boldsymbol{p}}$ fundido [mm]	(219,91; −200,05; 20,00)	
erro $\|\hat{\boldsymbol{p}} - \boldsymbol{p}^\star\|_{xy}$ / $\|\Delta z\|$ [mm]	0,108 / 0,000	

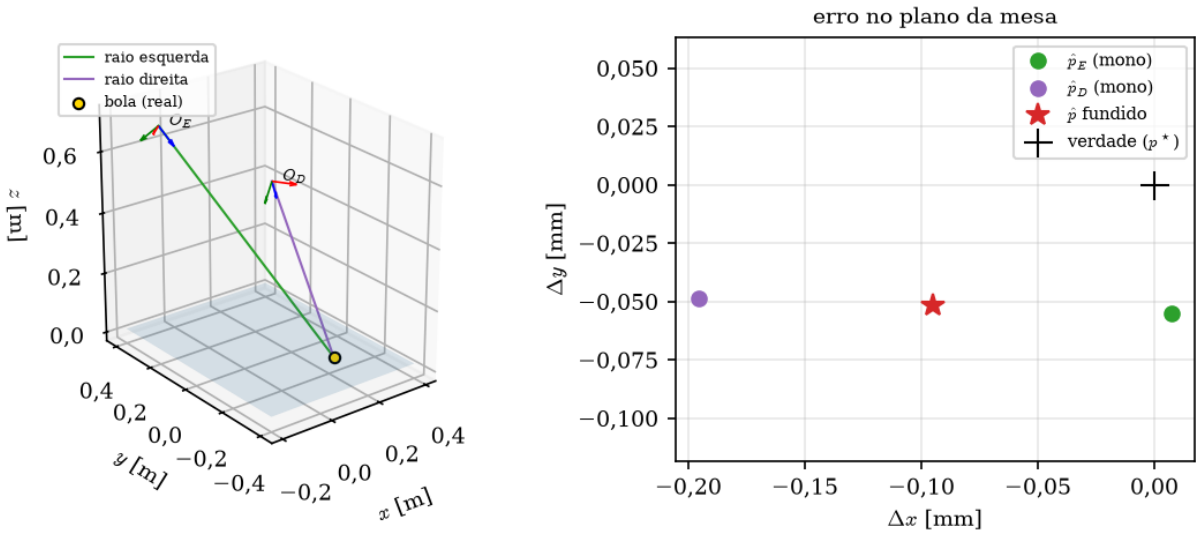


Figura 8: Esquerda: raios de retroprojeção das duas câmeras (eixos x_C, y_C, z_C em vermelho, verde e azul) até o plano Π (OM-X). Direita: erro das estimativas monoculares e da fundida no plano da mesa, em milímetros.

8 Análise de erro

8.1 Jacobiano da elevação

Proposição 7. *Seja $\boldsymbol{r}_k$ a k-ésima coluna de R. A derivada do ponto (15) em relação ao pixel é*

$$\frac{\partial \hat{\boldsymbol{p}}}{\partial u} = \frac{\lambda^\star}{f_x}\Big(\boldsymbol{r}_1 - \frac{r_{1z}}{d_z}\boldsymbol{d}\Big), \qquad \frac{\partial \hat{\boldsymbol{p}}}{\partial v} = \frac{\lambda^\star}{f_y}\Big(\boldsymbol{r}_2 - \frac{r_{2z}}{d_z}\boldsymbol{d}\Big), \tag{17}$$

com componente z nula. Definimos $J = [\partial_u \hat{\boldsymbol{p}} \;\; \partial_v \hat{\boldsymbol{p}}]_{xy} \in \mathbb{R}^{2\times 2}$.

Demonstração. De (13), $\partial \boldsymbol{d}/\partial u = \boldsymbol{r}_1/f_x$. De (15), $\partial\lambda^\star/\partial u = -\lambda^\star(\partial d_z/\partial u)/d_z = -\lambda^\star r_{1z}/(f_x d_z)$. Pela regra do produto, $\partial\hat{\boldsymbol{p}}/\partial u = (\partial\lambda^\star/\partial u)\boldsymbol{d} + \lambda^\star \partial\boldsymbol{d}/\partial u$, que é a primeira expressão. A componente z é $\frac{\lambda^\star}{f_x}(r_{1z} - r_{1z}) = 0$. O caso de v é análogo. □

No exemplo, em mm/px,

$$J_E = \begin{bmatrix} -0{,}7199 & -0{,}8043 \\ -0{,}6441 & 1{,}0072 \end{bmatrix}, \qquad J_D = \begin{bmatrix} 0{,}4374 & -0{,}7859 \\ -0{,}6759 & -0{,}4309 \end{bmatrix},$$

com valores singulares máximos 1,291 e 0,905 mm/px. A fórmula (17) concorda com diferenças finitas centradas até $3{,}8 \times 10^{-13}$ m/px. A Figura 9 mapeia $\sigma_{\max}(J)$ sobre a mesa. A sensibilidade cresce com a distância e com a obliquidade do raio, e a fusão com pesos iguais reduz a covariância.

8.2 Propagação de covariância e verificação de Monte Carlo

Se o erro do centro em cada vista é independente, com covariância $\sigma^2 I_2$, a linearização dá

$$\Sigma_i \approx \sigma^2 J_i J_i^{\mathsf{T}}, \qquad \Sigma_{\hat{\boldsymbol{p}}} \approx \sum_i w_i^2 \, \Sigma_i. \tag{18}$$

Para verificar (18), somamos ruído gaussiano com $\sigma = 0{,}5$ px aos centros detectados do exemplo, repetimos elevação e fusão 20.000 vezes e comparamos a covariância empírica com a prevista. Os semieixos do elipsoide de 1σ foram (0,329; 0,382) mm (empírico) e (0,327; 0,380) mm (previsto), com diferença relativa de Frobenius de 0,010 (OMY: (0,502; 0,603) contra (0,500; 0,600) mm). A linearização é, portanto, excelente nessa escala, e (18) poderia preencher o campo `covariance` do `BallObservation`.

8.3 Erro no modelo do plano

Se o centro real está em $z_\Pi + \delta z$ (raio configurado errado, mesa mais alta ou bola sobre um apoio), o raio que passa pelo centro real encontra Π em $\boldsymbol{p}^\star - (\delta z/d_z)\boldsymbol{d}$. O erro horizontal é então

$$\Delta\hat{\boldsymbol{p}}_{xy} = -\delta z \, \frac{\boldsymbol{d}_{xy}}{d_z}, \qquad \|\Delta\hat{\boldsymbol{p}}_{xy}\| = |\delta z| \tan\theta, \tag{19}$$

com θ o ângulo do raio com a vertical. No exemplo, $\tan\theta_E = 0{,}889$ e $\tan\theta_D = 0{,}521$. Na fusão, os deslocamentos são vetoriais e se cancelam em parte, porque as câmeras estão em lados opostos: o erro fundido é 0,578 mm por milímetro de δz (OMY: 0,651). Por exemplo, configurar o raio da bola com 1 mm de erro desloca a estimativa em ≈ 0,6 mm.

8.4 Erro de calibração

Uma pequena rotação $\delta\phi$ da câmera desvia todos os raios do mesmo ângulo, o que equivale, em primeira ordem, a um deslocamento de pixel de $f\,\delta\phi$. Com $f \approx 938$ px e $\sigma_{\max}(J) \approx 1{,}3$ mm/px, 1 mrad de erro de orientação vale ≈ 0,94 px ≈ 1,2 mm. Um erro de translação $\delta\boldsymbol{O}$ desloca $\hat{\boldsymbol{p}}$ em $\delta\boldsymbol{O}_{xy} - \delta O_z \, \boldsymbol{d}_{xy}/d_z$. Em simulação, os dois são nulos até 10^{-10} (Seção 2). Numa célula real, a calibração extrínseca tende a ser a fonte de erro dominante.

8.5 Oclusão parcial: estudo de caso

O caso mais desfavorável da varredura é a pose `mid_backish` do OM-X, com a bola em $(0{,}16;\ -0{,}12)$ m (Figura 10). Na câmera esquerda, o braço esconde a maior parte da bola: só 12,1 % da silhueta teórica está visível. A circularidade da parte visível é 0,51, acima de $c_{\min} = 0{,}45$, e a detecção é aceita. O MEC cai no regime (ii) da Proposição 5. Com um oclusor reto, o modelo preveria $h \approx 14$ px para essa fração visível; o erro de centro medido é 16,28 px (borda do braço não é reta). A estimativa monocular esquerda erra 17,61 mm, enquanto a direita, sem oclusão, erra 0,14 mm. Como a circularidade baixa dá peso $w_E = 0{,}363$, a fusão atenua, mas não elimina, o erro: o resultado fundido erra 6,40 mm. Isso ainda fica dentro da tolerância de 15 mm do benchmark, mas mostra o ponto fraco do esquema de pesos (Seção 11).

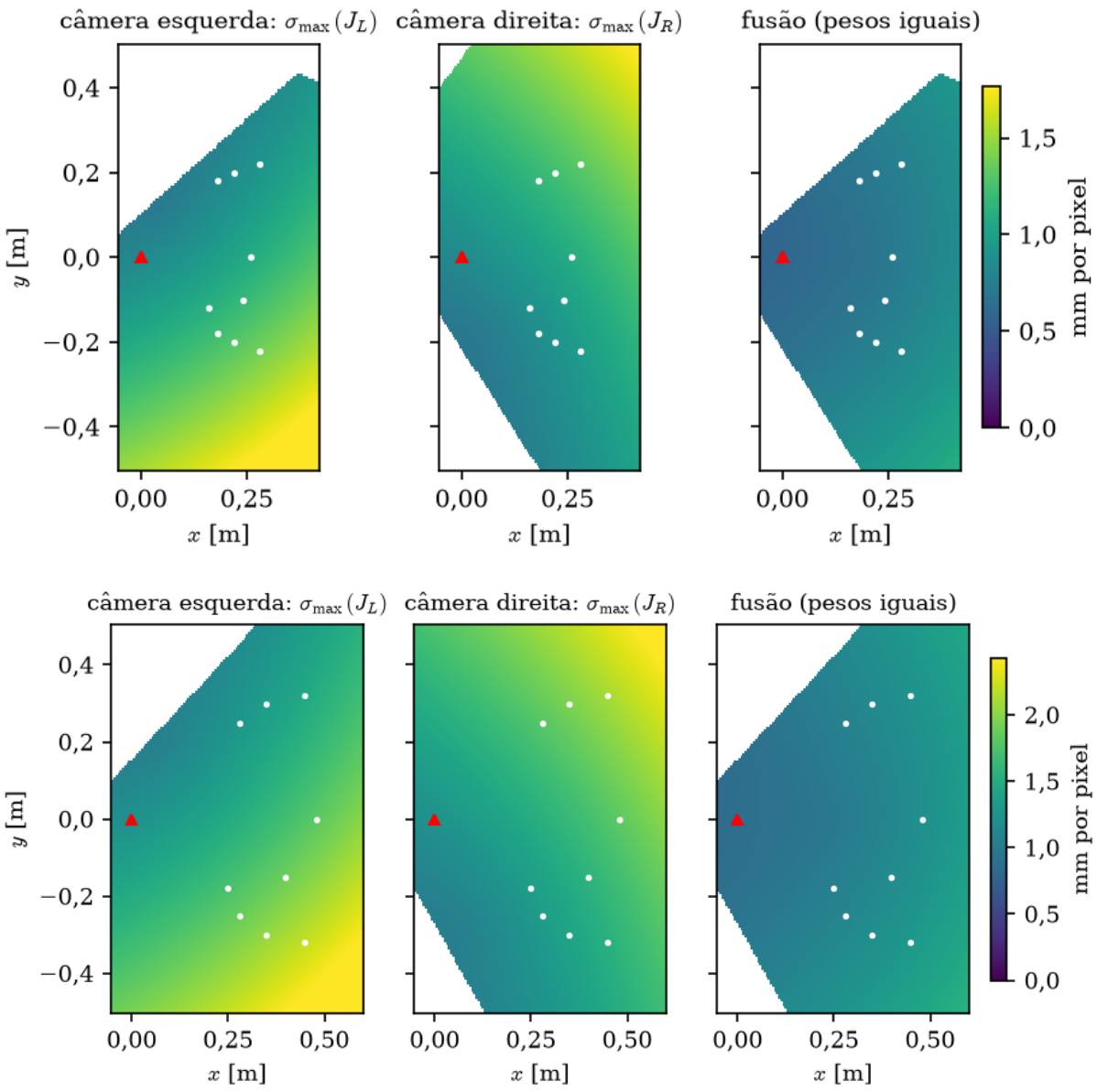


Figura 9: Sensibilidade $\sigma_{\max}(J)$ do ponto elevado ao erro de pixel (mm/px) sobre o plano Π, só onde o ponto projeta dentro da imagem. Acima: OM-X; abaixo: OMY. Pontos brancos: poses da varredura da Seção 9; triângulo: base do braço. O mapa ignora oclusão pelo robô.

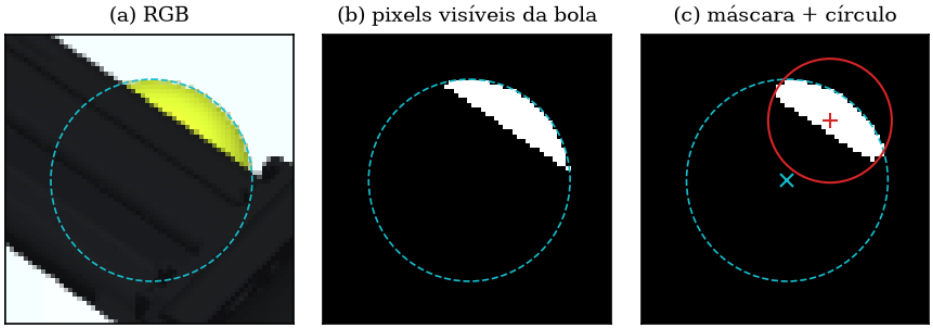


Figura 10: Oclusão parcial pelo braço (OM-X, `mid_backish`, câmera esquerda). Tracejado ciano: silhueta teórica completa (Proposição 3); ×: projeção exata do centro; vermelho: MEC da máscara e centro estimado.

9 Resultados na simulação

As Tabelas 2 e 3 repetem as nove poses de `scripts/benchmark_mujoco_vision.py`. Para cada câmera, a coluna "visível" é a razão entre os pixels da bola na segmentação de referência e a área teórica $\pi f^2 ab$ da silhueta. Ela mede diretamente a oclusão.

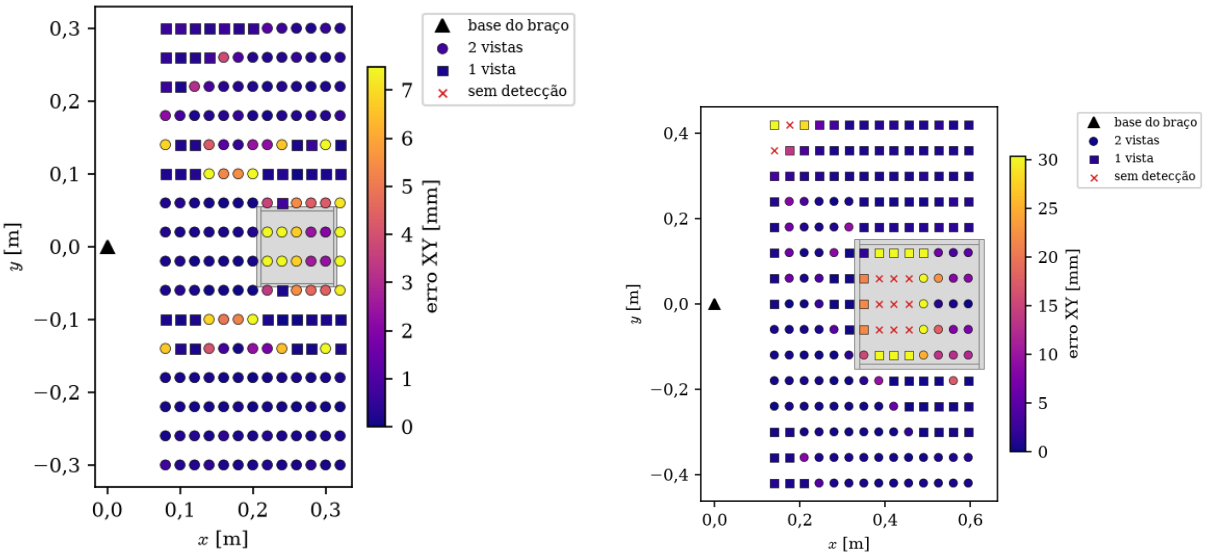
Tabela 2: Varredura OM-X (E / D = câmera esquerda / direita). Erro XY da estimativa fundida em relação à posição real da bola no MuJoCo.

pose	(x, y) [m]	vistas	visível [%]	c	erro centro [px]	erro XY [mm]
`pick_default`	(0,22; −0,20)	2	99 / 99	0,89 / 0,90	0,05 / 0,22	0,11
`pick_opposite`	(0,22; 0,20)	2	99 / 99	0,90 / 0,90	0,40 / 0,24	0,22
`front_cone`	(0,26; 0,00)	2	66 / 66	0,82 / 0,79	3,72 / 2,32	2,88
`near_base_mY`	(0,18; −0,18)	2	99 / 99	0,89 / 0,89	0,07 / 0,15	0,09
`near_base_pY`	(0,18; 0,18)	2	99 / 99	0,89 / 0,90	0,85 / 0,31	0,50
`far_mY`	(0,28; −0,22)	2	99 / 99	0,91 / 0,89	0,11 / 0,15	0,12
`far_pY`	(0,28; 0,22)	2	99 / 99	0,89 / 0,90	0,31 / 0,05	0,14
`mid_front`	(0,24; −0,10)	1	0 / 99	– / 0,89	– / 0,02	0,02
`mid_backish`	(0,16; −0,12)	2	12 / 99	0,51 / 0,89	16,28 / 0,15	6,40

Tabela 3: Varredura OMY. Em `front_cone`, a bola está dentro da caixa de depósito (paredes de 0,28 m); só uma calota de cerca de 11 % aparece por cima da parede, e ela não passa nos filtros de área e circularidade. O pipeline devolve `None` e não inventa uma pose.

pose	(x, y) [m]	vistas	visível [%]	c	erro centro [px]	erro XY [mm]
`pick_default`	(0,35; −0,30)	2	99 / 99	0,90 / 0,89	0,27 / 0,52	0,42
`pick_opposite`	(0,35; 0,30)	1	99 / 15	0,90 / –	0,66 / –	0,75
`front_cone`	(0,48; 0,00)	0	11 / 11	– / –	– / –	–
`near_base_mY`	(0,28; −0,25)	2	99 / 99	0,90 / 0,90	0,23 / 0,46	0,45
`near_base_pY`	(0,28; 0,25)	2	99 / 48	0,90 / 0,59	0,68 / 1,31	0,55
`far_mY`	(0,45; −0,32)	2	78 / 99	0,83 / 0,89	0,22 / 0,33	0,39
`far_pY`	(0,45; 0,32)	1	99 / 0	0,90 / –	0,87 / –	1,16
`mid_front`	(0,40; −0,15)	1	0 / 59	– / 0,77	– / 2,78	3,69
`mid_backish`	(0,25; −0,18)	2	99 / 99	0,91 / 0,89	0,27 / 0,23	0,32

Para uma visão espacial, a Figura 11 varre a bola numa grade densa: 208 posições no OM-X e 210 no OMY, com o braço na configuração de sensoriamento. Quando as duas câmeras veem mais de 95 % da silhueta (93 posições no OM-X e 38 no OMY), o erro XY tem mediana 0,14 mm e máximo 0,73 mm (OM-X), e mediana 0,43 mm e máximo 0,70 mm (OMY), de acordo com a análise de pixel da Seção 8. Nas demais posições, os erros se concentram em faixas bem definidas: onde o braço ou as paredes da caixa cobrem parte da bola numa das vistas (regime (ii) da Proposição 5). Na grade completa, a mediana é 0,25 mm e o percentil 95 é 7,49 mm (OM-X); no OMY, a mediana é 0,75 mm e o percentil 95 é 30,37 mm. O OM-X detecta a bola em todas as 208 posições, e o OMY em 199 das 210. As falhas do OMY ficam dentro da caixa de depósito ou no canto $x \approx 0{,}15$ m, $y \approx 0{,}4$ m, onde nenhuma câmera vê a bola sem obstrução.



(a) OM-X **(b)** OMY

Figura 11: Erro XY da estimativa fundida numa grade densa de posições da bola. Círculo: duas vistas; quadrado: uma vista; ×: sem detecção. Cinza: caixa de depósito. A escala de cor satura no percentil 95.

10 Da posição da bola à pose de preensão

10.1 Por que a posição basta

Uma esfera homogênea é invariante por qualquer rotação em torno do próprio centro. A ação de $SE(3)$ sobre a esfera tem estabilizador $SO(3)$ (rotações em torno do centro), e o espaço de poses distinguíveis é o quociente $SE(3)/SO(3) \cong \mathbb{R}^3$. Portanto, a "pose" do objeto é exatamente a posição $\hat{\boldsymbol{p}}$ do centro, e é isso que o `BallObservation` carrega. A orientação da garra não é imposta pela visão: ela fica livre e é escolhida pela cinemática inversa.

10.2 Alvos cartesianos

A partir de $\hat{\boldsymbol{p}}$, `_pad_mid_pick_waypoints` (`fret/control/pick_place_vision.py`) define três alvos para o ponto médio entre as pontas dos dedos:

$$\boldsymbol{p}_{\text{hover}} = \hat{\boldsymbol{p}} + h_{\text{hover}}\boldsymbol{e}_z, \qquad \boldsymbol{p}_{\text{grasp}} = \hat{\boldsymbol{p}} + 0{,}012\,\boldsymbol{e}_z, \qquad \boldsymbol{p}_{\text{lift}} = \hat{\boldsymbol{p}} + h_{\text{lift}}\boldsymbol{e}_z, \tag{20}$$

com $(h_{\text{hover}}, h_{\text{lift}}) = (0{,}08;\ 0{,}12)$ m no OM-X e $(0{,}10;\ 0{,}14)$ m no OMY. O deslocamento de 12 mm acima do centro na preensão faz os dedos fecharem sobre a esfera sem tocar a mesa. A abertura da garra é fixada por alvo: aberta em *hover* e fechada ("pinça") em *grasp* e *lift*.

10.3 Cinemática inversa do ponto médio dos dedos

Seja $\boldsymbol{m}(\boldsymbol{q}) = \frac{1}{2}(\boldsymbol{x}_{\text{R}}(\boldsymbol{q}) + \boldsymbol{x}_{\text{L}}(\boldsymbol{q}))$ o ponto médio dos centros das geometrias `pad_right` e `pad_left`, calculado pela cinemática direta do MuJoCo, com $\boldsymbol{q} \in \mathbb{R}^n$ ($n = 4$ no OM-X e $n = 6$ no OMY). Para cada alvo $\boldsymbol{p}_*$, resolve-se

$$\boldsymbol{q}_* = \underset{\boldsymbol{q} \in [\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}]}{\arg\min} \ \|\boldsymbol{m}(\boldsymbol{q}) - \boldsymbol{p}_*\|^2 \tag{21}$$

pelo método L-BFGS-B [7] (`pad_mid_ik`, `fret/control/omy_pad_mid_ik.py`), a partir de várias sementes: a solução do alvo anterior (*warm start*), as configurações do YAML do cenário e sementes específicas do robô. Fica a solução de menor resíduo. Se o resíduo $\|\boldsymbol{m}(\boldsymbol{q}_*) - \boldsymbol{p}_*\|$ passar de 8 mm, a rotina levanta erro em vez de enviar uma preensão ruim. Como o OM-X tem 4 graus

de liberdade e o alvo tem 3 dimensões, (21) tem em geral uma família de soluções; a escolha entre elas vem das sementes. No exemplo da Tabela 1, os resíduos foram 0,004 mm (*hover*), 0,000 mm (*grasp*) e 0,003 mm (*lift*) no OM-X, e $\leq$ 0,001 mm no OMY. Portanto, o erro de posicionamento cartesiano é dominado pela visão, não pela IK.

10.4 Integração com a máquina de estados

Nos simuladores (`simulate_pick_place` e `simulate_omy_pick_place`), a visão é chamada uma vez, antes de o braço dobrar para a postura de repouso. A ordem importa porque a postura dobrada oclui uma das câmeras. `apply_vision_pick_goals` renderiza as câmeras, executa o pipeline e, se não houver `BallObservation`, levanta erro: o sistema não pega às cegas. Com a observação, a função substitui as configurações de coleta da máquina de estados (as de depósito continuam vindo do YAML, pois o depósito é um *fixture* conhecido) e posiciona a esfera visual `cv_ball_ghost` em $\hat{\boldsymbol{p}}$ (Figura 3).

11 Limitações e melhorias possíveis

1. **Associação entre vistas.** Cada câmera escolhe seu próprio *blob* de maior circularidade, e nada verifica se as vistas viram o mesmo objeto. Com duas bolas visíveis, a Eq. (16) pode fazer a média de dois objetos diferentes e produzir um ponto entre eles. Um teste de consistência $\|\hat{\boldsymbol{p}}_E - \hat{\boldsymbol{p}}_D\| \leq \tau$ (por exemplo, $\tau = 3\sqrt{\mathrm{tr}(\Sigma_E + \Sigma_D)}$) resolveria esse caso.

2. **Oclusão parcial.** A circularidade só reduz o peso de uma vista oclusa (Seção 8.5). Algumas alternativas: rejeitar vistas com $\hat{\rho}$ muito menor que o raio previsto $f\,a(\hat{\boldsymbol{p}})$ (o Corolário 4 dá a previsão a partir da estimativa da outra vista), usar pesos de variância mínima com Σ_i inflada por $1/c_i$, ou aplicar o mesmo teste de consistência do item anterior.

3. **Viés de perspectiva.** É de no máximo $\approx 1{,}4$ px e pode ser corrigido de forma fechada pela Eq. (12).

4. **Hipótese do plano.** A estimativa de z é imposta. Uma bola fora da mesa (sobre um obstáculo ou em movimento no ar) gera erro horizontal de $\delta z \tan\theta$. Com duas vistas, uma triangulação por mínimos quadrados poderia estimar z e servir de diagnóstico.

5. **Cor e iluminação.** O limiar HSV é fixo. Funciona bem na simulação (Figura 4), mas com câmeras reais precisa de calibração de cor ou de um modelo de distorção de lente, ausente hoje.

6. **Incerteza.** O campo `covariance` pode ser preenchido com a Eq. (18), validada pelo experimento de Monte Carlo.

A Parâmetros das configurações

parâmetro	OM-X (`omx_portal_overhead.yml`)	OMY (`omy_portal_overhead.yml`)
resolução	1280×720	1280×720
`fovy` / f [px]	42° / 937,8321	45° / 869,1169
centros ópticos $\boldsymbol{O}$ [m]	$(-0{,}12;\ \pm 0{,}30;\ 0{,}70)$	$(-0{,}18;\ \pm 0{,}42;\ 0{,}98)$
$\boldsymbol{\ell}$ / $\boldsymbol{h}$ (HSV)	$(20; 40; 40)$ / $(50; 255; 255)$	$(20; 40; 40)$ / $(50; 255; 255)$
elemento estruturante	elíptico 5×5	elíptico 5×5
$A_{\min}$ / $A_{\max}$ [px^2]	80 / 500.000	120 / 500.000
$c_{\min}$	0,45	0,45
z_{mesa} / r [m]	0 / 0,020	0 / 0,043

Tabela 4: Parâmetros do pipeline nas duas células com pórtico. Em `config/vision/` também há `hsv_blob_overhead.yml` (fixture sintético, 640×480, $c_{\min} = 0{,}55$) e `hsv_blob_tennis_yellow.yml` (fotos da web).

B Reprodutibilidade

Todas as figuras, tabelas e números deste documento são gerados por

```
MUJOCO_GL=egl PYOPENGL_PLATFORM=egl python3 scripts/vision_algorithm_figures.py
cd docs/vision/algoritmo-visao && latexmk -pdf algoritmo_visao.tex
```

O script usa o pipeline de produção (`pipeline_from_hsv_plane_config`) e verifica por asserção que a reimplementação passo a passo coincide com ele: máscara idêntica, mesmo centro de detecção e posição fundida igual até 10^{-12} m. As métricas brutas ficam em `metricas.json`.

Referências

[1] R. Hartley e A. Zisserman. *Multiple View Geometry in Computer Vision*, 2. ed. Cambridge University Press, 2004.

[2] J. Serra. *Image Analysis and Mathematical Morphology*. Academic Press, 1982.

[3] S. Suzuki e K. Abe. Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing*, 30(1):32–46, 1985.

[4] E. Welzl. Smallest enclosing disks (balls and ellipsoids). In *New Results and New Trends in Computer Science*, LNCS 555, p. 359–370. Springer, 1991.

[5] A. R. Smith. Color gamut transform pairs. *ACM SIGGRAPH Computer Graphics*, 12(3):12–19, 1978.

[6] H. Freeman. On the encoding of arbitrary geometric configurations. *IRE Trans. Electronic Computers*, EC-10(2):260–268, 1961.

[7] R. H. Byrd, P. Lu, J. Nocedal e C. Zhu. A limited memory algorithm for bound constrained optimization. *SIAM J. Sci. Comput.*, 16(5):1190–1208, 1995.

[8] G. Bradski. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 2000.